

A Project Report on StockSmart

Submitted by: Avash Shrestha,
Diya Manandhar, Krishala Pokhrel, Shreyas Regmi

Roll No: 24080271, 24080277
24080285, 24080300

Submitted to
Mr. Sanju Shrestha
Ace Institute of Management
Pokhara University

In partial fulfillment of the requirement for the degree of Bachelors of Computer System
and Information Technology

Kathmandu 22 June, 2026

Supervisor Recommendation

This is to certify that this project entitled, “**StockSmart**” prepared and submitted by **Diya Manandhar, Krishala Pokhrel, Avash Shrestha** and **Shreyas Regmi** in partial fulfillment of the requirements of the degree of Bachelor of Computer System and Information Technology (BCSIT) awarded by Pokhara University, has been completed under my supervision. I recommend the same for acceptance by the University.

.....

Supervisor

Mr. Sanju Shrestha

Ace Institute of Management

Certificate of Approval

This project entitled, “**StockSmart**” prepared and submitted by **Diya Manandhar, Krishala Pokhrel, Avash Shrestha** and **Shreyas Regmi** have been examined and is accepted for the award of the degree of Bachelor of Computer System and Information Technology (BCSIT) awarded by Pokhara University.

.....

Supervisor

Mr. Sanju Shrestha

Ace Institute of Management

.....

Examiner

.....

Associate Director

Mr. Rajat Bajracharya

Ace Institute of Management

.....

Director

Mr. Ramesh Kumar Chauhan

Ace Institute of Management

Abstract

Stock Smart is an open-source inventory management system designed to help small and medium-sized businesses efficiently manage their inventory, sales, and stock-related operations. The system provides a centralized platform for tracking products, monitoring stock levels, recording transactions, and generating inventory reports in real time. By automating routine inventory tasks, Stock Smart minimizes human error, reduces stock discrepancies, and improves operational efficiency.

The project follows an open-source approach, allowing developers and organizations to customize, extend, and deploy the system according to their specific requirements. The system incorporates role-based access control, enabling different users such as administrators, inventory managers, warehouse staff, and cashiers to perform tasks relevant to their responsibilities. Additional features include product categorization, stock movement tracking, low-stock alerts, sales monitoring, and data analytics to support informed decision-making.

Stock Smart aims to provide an affordable and scalable alternative to proprietary inventory management solutions while encouraging community-driven development and continuous improvement. Through its user-friendly interface and flexible architecture, the system enhances inventory visibility, streamlines business processes, and contributes to better resource management for organizations of varying sizes.

Acknowledgement

We would like to express our sincere gratitude to our project supervisor, Mr. Sanju Shrestha, for providing help with all kinds of technical support, encouragement, and valuable suggestions throughout the development of this project. We would also like to thank fellow classmates whose projects inspired us with ideas for our own. Lastly, we would like to thank Pokhara University for providing us with the opportunity and resources to undertake this project as part of the Bachelor in Computer System and Information Technology (BCSIT) program.

Table of Content

Contents:

Supervisor Recommendation.....	i
Certificate of Approval.....	ii
Abstract.....	iii
Acknowledgement.....	iv
Abbreviations.....	vii
List of Figures.....	viii
Chapter 1: Introduction.....	1
1.1 Background of the Study.....	1
1.2 Statement of the Problem.....	2
1.2.1 Defining the Problem.....	2
1.2.2 Who Is Affected.....	2
1.2.3 Impact of the Problem.....	3
1.3 Objectives.....	3
1.3.1 Main Objective.....	3
1.3.2 Specific Objectives.....	3
1.4 Scope of the Study.....	4
1.5 Limitations of the Study.....	5
Chapter 2: Review of Literature.....	7
2.1 Description of fundamental theories, general concepts and terminologies related to the project.....	7
2.2 Review of the similar Projects and Theories.....	8
Chapter 3: System Analysis and Design.....	11
3.1 System Analysis.....	11
3.1.1 Requirement Analysis.....	11
3.1.2. Feasibility Analysis.....	12

3.1.3 Data Modeling.....	14
3.1.4 Process Modeling.....	15
3.2 System Design.....	16
3.2.1 Architectural Design.....	16
3.2.2 Database Schema Design.....	23
3.2.3 Interface Design.....	23
3.2.4 Physical Model.....	24
Chapter 4: Implementation and Testing.....	25
4.1 Implementation.....	25
4.1.1 Tools Used.....	25
4.1.2. Test Cases for system Testing.....	26
Chapter 5: Conclusion and Future Recommendations	27
5.1 Lessons Learned	27
5.2. Conclusion.....	27
5.3. Recommendations.....	27
Appendices.....	29
References.....	33

Abbreviations

AI: Artificial Intelligence

API: Application Programming Interface

BCSIT: Bachelor of Computer System and Information Technology

ERP: Enterprise Resource Planning

IoT: Internet of Things

LAMP: Linux, Apache, MySQL, PHP

P.U.: Pokhara University

QC: Quality Control

QR: Quick Response

RFID: Radio Frequency Identification

SKU: Stock Keeping Unit

SME: Small and Medium Enterprise

SMEs: Small and Medium Enterprises

UI: User Interface

USD: United States Dollar

UX: User Experience

VPS: Virtual Private Server

List of Figures

- Figure 1: Use-case Diagram
- Figure 2: Flowchart Cashier
- Figure 3: Flowchart Warehouse
- Figure 4: Flowchart Inventory Management
- Figure 5: Flowchart Admin
- Figure 6: ER Model
- Figure 7: Product Page
- Figure 8: Inventory Page
- Figure 9: Checkout Page
- Figure 10: Gantt Chart

Chapter 1: Introduction

1.1 Background of the Study

Any business that sells physical goods will eventually have to face the same question: how do you know what you have, and how do you know when you are about to run out? It seems simple enough. But for many small and medium-sized businesses across Nepal and around the world, this question does not have a good answer. Not because the owners do not care, but because the tools available to them have either been too expensive, too complicated, or simply not designed with their kind of business in mind.

Inventory management, at its most basic, refers to keeping track of stock from the time it arrives at a business until it leaves as a sold product. It means knowing how many units of each product are on the shelf, when the stock is running low, what has expired or is about to expire.

how much was sold over a given period. If done well, it keeps a business running smoothly. If done poorly, or not done at all, it leads to empty shelves, wasted stock, and decisions made on guesswork.

The scale of this problem across the SME sector is difficult to ignore. Industry data shows that 43 percent of small and medium enterprises experience stock-outs every month. That means nearly half of all SMEs are regularly unable to meet customer demand simply because they did not know their stock had dropped that low. Globally, poor inventory control is estimated to cost businesses around USD 1.1 trillion per year. About 30 percent of inventory losses among SMEs come from expired or wasted goods. And two in every three SMEs are still tracking their stock by hand, either on paper or in a basic spreadsheet.

Behind each of these numbers is a real business: a shop owner who lost a customer because a product was out of stock, a warehouse manager who discovered an inconsistency during a count that took two days to resolve, a supermarket that had to dispose of a batch of goods that nobody noticed was approaching expiry. StockSmart began as a response to these very real and very common situations.

1.2 Statement of the Problem

1.2.1 Defining the Problem

Many small and medium businesses today operate without a clear picture of their own inventory. They may have a rough sense of what is on the shelf, shaped by experience and habit, but that is not the same as knowing. And when business decisions, from reordering to pricing to managing waste, depend on knowing what is actually in stock, that difference matters more than it might appear.

The core issue is this: SMEs lack access to a system that gives them accurate, up-to-date inventory information in a form that is simple to use and affordable to run. In the absence of such a system, they default to manual methods, each with their own failure modes. Paper records get lost or damaged. Spreadsheets are only as current as the last person who updated them. Memory is unreliable, especially when a business grows or staff changes.

These are not isolated incidents. They are the expected outcomes of managing inventory without a proper system. The problems that follow, from unexpected shortages to overfilled storerooms to products quietly expiring on a shelf, are predictable consequences of that original gap. And because they tend to happen gradually, they can go unnoticed until the financial damage is already done.

1.2.2 Who Is Affected

The businesses most affected by inadequate inventory management are those that deal in physical goods, have limited administrative resources, and cannot absorb the kinds of losses that poor stock control produces. In practice, that covers a wide range of SMEs.

- Small retail shops and convenience stores carry the full burden of stock management with very little support. There is no dedicated inventory team, no automated reorder system, and often no clear separation between the person selling and the person managing stock. Accurate manual record-keeping in that environment is difficult to sustain consistently.
- Warehouses that handle large and varied product ranges face a different problem. The absolute volume of goods moving through a warehouse means that manual tracking is not just inconvenient but genuinely unreliable. Errors that would be minor in a small shop compound quickly at warehouse scale.
- Supermarkets and grocery stores have the additional pressure of managing

perishable goods. Expiry date oversight is not just a financial concern; it has direct implications for customer safety and regulatory compliance. Manual checks, however conscientious, are not a dependable safeguard at scale.

- Wholesale distributors must plan their stock levels around demand patterns that shift with seasons and events. Without reliable inventory data, that planning becomes guesswork, and the consequences of guessing wrong, whether that means unsold stock or unfulfilled orders, are felt directly in the business's bottom line.

The effects of these problems also extend to customers. When a business runs out of stock, the customer goes without. When expired goods reach the shelf, the customer is the one put at risk. Poor inventory management is ultimately a failure that travels all the way through the supply chain to the person on the other end.

1.2.3 Impact of the Problem

The consequences of not managing inventory properly tend to fall into five recurring patterns. Each one is worth examining in some detail, because together they illustrate why this problem, though it might seem mundane, carries significant costs.

1.3 Objectives

This section presents the objectives that guide the design and development of StockSmart. The objectives define the intended outcomes of the system and serve as measurable benchmarks for evaluating its success upon completion.

1.3.1 Main Objective

The main objective of this project is to design and develop StockSmart, a simple and centralized web-based inventory management system for small and medium-sized enterprises (SMEs). The system aims to enable real-time inventory tracking, automate stock-related alerts, and streamline transaction recording so that businesses can reduce inventory losses and improve their overall operational efficiency.

1.3.2 Specific Objectives

In order to achieve the main objective stated above, the following specific objectives have been identified:

1. To develop a real-time inventory tracking module that allows authorized users to monitor current stock levels and record all stock-in and stock-out transactions accurately and efficiently.

2. To implement an automated low stock alert system that notifies relevant users when product quantities fall below a predefined minimum threshold, enabling timely restocking decisions.
3. To develop an automated expiry date monitoring system that tracks the expiry dates of perishable products and generates alerts before products expire, thereby reducing losses from expired or wasted goods.
4. To reduce financial losses caused by overstocking, stock shortages, and the disposal of expired products by providing management tools that support more informed and proactive inventory decisions.
5. To improve the accuracy and operational efficiency of inventory management by replacing manual processes with an automated, centralized system that minimizes human error and reduces the time spent on routine inventory tasks.
6. To lower operational overhead costs for SMEs through automation, self-hosted deployment, and the use of open-source technologies, making the system accessible to businesses with limited financial resources.

1.4 Scope of the Study

The scope of Stock Smart is focused on the development of an open-source inventory management system designed to support the inventory-related operations of small and medium-sized enterprises (SMEs). The system aims to provide a centralized platform for managing products, monitoring stock levels, recording sales transactions, and generating inventory reports in real time.

The project covers the core functionalities required for effective inventory management. These functionalities include product registration and categorization, stock tracking, inventory updates, stock movement monitoring, sales recording, and low-stock notifications. Through these features, businesses can maintain accurate inventory records and make informed decisions regarding stock replenishment and resource allocation.

Stock Smart incorporates role-based access control to ensure that different categories of users can access only the functionalities relevant to their responsibilities. Administrators can manage users and system settings, inventory managers can monitor stock activities, warehouse personnel can update stock records, and cashiers can process sales transactions. This approach enhances security and operational efficiency within the organization.

The system also provides reporting and analytical features that allow users to review inventory performance, identify fast-moving and slow-moving products, and monitor stock trends over time. Such information assists businesses in improving inventory planning and reducing losses caused by stock shortages or excess inventory.

As an open-source solution, the project is designed to be scalable and customizable. Organizations and developers can modify and extend the system according to their specific requirements. The system is intended for deployment in retail stores, supermarkets, warehouses, pharmacies, and other businesses that require efficient inventory control.

The project scope is limited to inventory and sales management functions and does not include advanced enterprise-level features such as supply chain management, accounting integration, payroll management, or customer relationship management systems.

1.5 Limitations of the Study

Despite its usefulness, Stock Smart has several limitations that should be considered. The system has been developed primarily for small and medium-sized enterprises and may not fully satisfy the complex inventory requirements of large-scale organizations with multiple branches and extensive supply chain networks.

The project focuses mainly on inventory and sales management functionalities. Features such as accounting integration, human resource management, supplier relationship management, and advanced business intelligence tools are beyond the scope of the current implementation. Organizations requiring these functionalities may need additional software solutions or future system enhancements.

The effectiveness of the system depends on the accuracy of data entered by users. Incorrect product information, stock updates, or transaction records may lead to inaccurate inventory reports and decision-making outcomes. Therefore, proper user training and data management practices are necessary for successful implementation.

The current version of the system may also have limitations regarding integration with third-party services such as payment gateways, barcode scanners, RFID technologies, and external ERP platforms. Although the open-source architecture allows future expansion, these integrations are not fully implemented within the present study.

Furthermore, the system has been tested within a limited development environment and may require additional testing under large-scale operational conditions. Factors such as

network reliability, server performance, and concurrent user access may influence system performance in real-world deployments.

Finally, due to time, resource, and academic project constraints, the study does not include long-term deployment evaluation or comprehensive user acceptance testing across multiple organizations.

Chapter 2: Review of Literature

2.1 Description of fundamental theories, general concepts and terminologies related to the project

Inventory management is the process of planning, organizing, monitoring, and controlling inventory throughout its lifecycle, from procurement to sale. Effective inventory management ensures that the right products are available in the right quantity at the right time while minimizing storage costs and inventory losses. For small and medium-sized enterprises (SMEs), proper inventory management is essential for maintaining operational efficiency, customer satisfaction, and profitability.

Inventory Management System (IMS)

An Inventory Management System (IMS) is a software solution designed to track, manage, and control inventory-related activities within an organization. It provides a centralized platform for recording stock movements, monitoring inventory levels, managing product information, and generating reports. By automating inventory processes, an IMS reduces human error, improves data accuracy, and supports better decision-making.

Stock Tracking

Stock tracking refers to the continuous monitoring of product quantities within an inventory. It records stock additions, sales, returns, transfers, and adjustments in real time. Accurate stock tracking helps businesses prevent stock shortages, reduce overstocking, and maintain reliable inventory records.

Real-Time Inventory Management

Real-time inventory management is the practice of updating inventory records immediately whenever stock-related transactions occur. This approach allows users to view current stock levels at any time and make timely decisions regarding purchasing, sales, and replenishment activities.

Stock Keeping Unit (SKU)

A Stock Keeping Unit (SKU) is a unique identifier assigned to each product in an inventory system. SKUs help businesses distinguish products based on characteristics such as category, size, brand, or variation. Proper SKU management improves inventory organization and tracking efficiency.

Inventory Control

Inventory control involves maintaining optimal inventory levels to meet customer demand while minimizing inventory holding costs. Effective inventory control helps organizations balance supply and demand, reduce waste, and improve resource utilization.

Low-Stock Alert System

A low-stock alert system automatically notifies users when the quantity of a product falls below a predefined threshold. Such alerts enable businesses to replenish stock before shortages occur, reducing the risk of lost sales and customer dissatisfaction.

Expiry Date Monitoring

Expiry date monitoring is particularly important for businesses dealing with perishable products. The process involves tracking product expiration dates and generating alerts before products expire. This helps reduce inventory waste, improve product quality, and ensure customer safety.

Role-Based Access Control (RBAC)

Role-Based Access Control (RBAC) is a security mechanism that restricts system access based on user roles and responsibilities. Different users, such as administrators, inventory managers, warehouse staff, and cashiers, are granted access only to the functionalities required for their tasks. This improves security and prevents unauthorized system operations.

Reporting and Data Analytics

Reporting and data analytics provide users with insights into inventory performance through summaries, charts, and statistical information. Inventory reports help businesses identify sales trends, monitor stock movement, evaluate product performance, and support strategic decision-making.

Open-Source Software

Open-source software refers to software whose source code is publicly available for modification, distribution, and improvement. The open-source nature of StockSmart allows organizations to customize the system according to their specific requirements while avoiding expensive licensing costs.

Conceptual Framework of StockSmart

The StockSmart system is based on the principle of centralized inventory management. Product information, inventory records, sales transactions, and user activities are stored within a unified database. Users interact with the system through role-specific interfaces, while the application automatically updates stock levels, generates alerts, and produces reports. This centralized approach improves inventory visibility, operational efficiency, and decision-making accuracy. This theoretical foundation supports the design and implementation of StockSmart as an effective inventory management solution for SMEs.

2.2 Review of the similar Projects and Theories

Paper 1: Web-Based Inventory Platform for SMEs (Source: Agboola, Malgwi, Mahmud & Oguntoye (2022))

Advantages:

- Helps replacing spreadsheets with digital records to reduce errors
- Includes product tracking, sales management, and user management modules
- Improved inventory accuracy and reduced operational cost for SMEs

Limitations:

- It may not work the same in all industries.
- Less flexible for changing business.
- It does not have automatic expiry alerts.

Paper 2: ERP Implementation for Supply Chain Management (Source: Qureshi (2022))

Advantages:

- Identifies the key factors for successfully implementing Enterprise Resource Planning, in supply chains and ranks them based on importance.
- It uses ISM and MICMAC methods to show how the key factors are connected.
- ERP systems help businesses grow and make supply chains better.

Limitations:

- ERP systems are extremely costly and complex which are impractical for small enterprises
- ERP systems take a long time to set up and require skilled IT staff
- Study focuses on only large organizations

Paper 3: IoT-Based Inventory System (Source: Ghulam Haider & Edward Lucas (2024))

Advantages:

- Machine learning-based inventory management system
- Uses forecasting (Prophet), pattern mining (Apriori), prediction (Random Forest)
- Achieves high accuracy in demand prediction (~94%)

Limitations:

- Requires large historical datasets
- High computational complexity
- Does not address expiry-date or batch-level tracking

Paper 4: AI-Based Inventory System (Source: Abdul Majid K & Kavyashree G.J (2025))

Advantages:

- IoT-based automated inventory tracking system
- Uses sensors for real-time stock monitoring
- Reduces manual inventory effort

Limitations:

- Requires IoT hardware (sensors/RFID)
- High setup and maintenance cost
- Technical complexity for SMEs

Chapter 3: System Analysis and Design

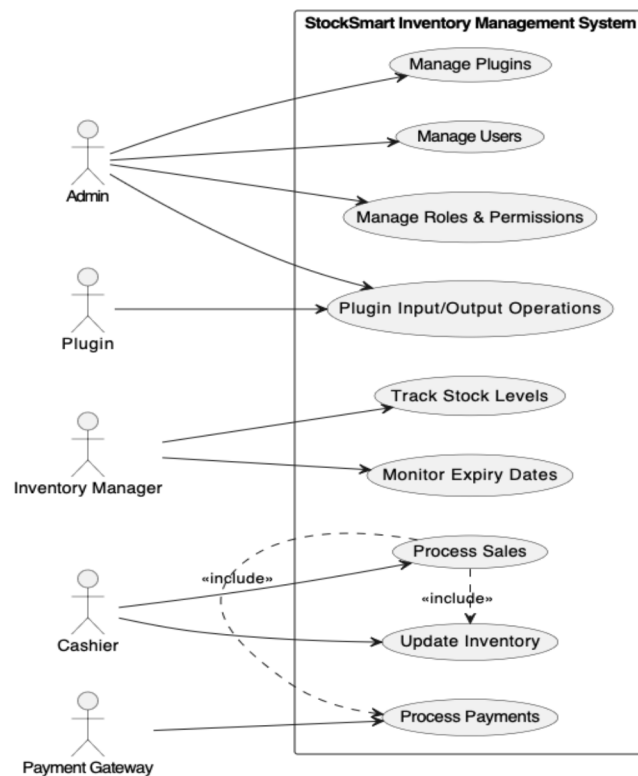
3.1 System Analysis

3.1.1 Requirement Analysis

The first step of the methodology involves a thorough analysis of the inventory-related challenges commonly faced by small and medium-sized enterprises. This includes examining problems arising from manual inventory practices, such as inaccurate stock counts, delayed responses to stockouts, and the absence of structured monitoring for product expiry. Based on this analysis, the key functional requirements of StockSmart are identified. These requirements cover core features such as real-time stock tracking, transaction recording, alert generation, user role management, and report generation. The outcome of this step provides a clear and agreed-upon foundation for all subsequent design and development work.

- Functional Requirement

Figure 1: Use-case Diagram



- Non-Functional Requirements

- 1) Performance

The system shall respond to user requests within an acceptable time frame. Inventory searches, report generation, and transaction processing shall be performed efficiently.

- 2) Reliability

The system shall maintain consistent operation and minimize system downtime. Data shall remain available and accurate throughout system usage.

- 3) Security

User authentication and authorization mechanisms shall be implemented to protect sensitive information. Passwords shall be securely stored, and role-based access control shall restrict unauthorized access.

- 4) Usability

The system shall provide an intuitive and user-friendly interface that can be used effectively by individuals with varying technical expertise.

- 5) Scalability

The system shall support future growth in terms of users, products, transactions, and additional functionalities.

- 6) Maintainability

The system shall be designed using modular components to facilitate future updates, bug fixes, and feature enhancements.

- 7) Portability

The system shall operate across different devices and modern web browsers with minimal compatibility issues.

3.1.2. Feasibility Analysis

- Technical

Technical feasibility refers to whether the project can be built using available technologies and within the technical capabilities of the development team. StockSmart will be developed using a well-established and widely documented technology stack. All selected technologies are supported by extensive official documentation, active developer communities, and a proven track record of use

in similar systems. The development team already possesses practical knowledge of a significant portion of this stack, reducing the risk of major technical obstacles during development. Where new tools or libraries are required, they are mature and well-supported, with readily available learning resources. Additionally, the overall scope of the system is appropriate for an SME-focused solution, meaning the implementation does not require unnecessarily complex infrastructure. Supporting functionalities such as alert notifications and report generation will be handled using established libraries and APIs, further reducing development complexity. On this basis, the project is considered technically feasible.

- **Operational**

Operational feasibility refers to whether the system can realistically function within the day-to-day operations of an SME and be used effectively by its intended users. StockSmart is designed with simplicity and ease of use as central priorities. The user interface will be straightforward and intuitive, requiring minimal technical training for staff to operate it confidently. Role-based access ensures that each user type, whether Admin, Inventory Manager, or Cashier, interacts only with the features relevant to their responsibilities, reducing the risk of errors and unnecessary complexity. Automated alerts for low stock and product expiry remove the need for constant manual monitoring, which directly lowers the operational effort required from staff. Furthermore, the planned integration with electronic paper-based price labels offers additional operational benefits for clients by enabling automatic price updates, which reduces manual labelling tasks and associated labour costs. The system is therefore considered operationally feasible.

- **Financial**

Economic feasibility refers to whether the system can be developed and maintained without incurring costs that are prohibitive for either the development team or the end users. StockSmart is built entirely using open-source software, which eliminates all software licensing fees. The system is designed to be self-hosted on the client's own hardware or local network, removing the need for paid cloud hosting services, a virtual private server (VPS), a public IP address, or an external

domain. This significantly reduces the ongoing operational costs that a business would typically incur when using a cloud-dependent system. For SMEs with sufficient technical capability, the open-source nature of the software means the system can be deployed and managed at virtually no additional cost. Payment processing functionality, where applicable, will be implemented using free and open-source tools, keeping transactional overheads minimal. Post-deployment maintenance will be carried out by the development team until the system is stable and all identified issues are resolved, ensuring that clients are not left without support after handover. Based on these considerations, the project is confirmed to be economically feasible, particularly for SMEs operating with limited budgets.

- **Schedule**

The development of Stock Smart was found to be schedule feasible as the project could be completed within the time frame allocated for the academic project. A detailed project plan was prepared at the beginning of the development process, dividing the work into several phases including requirement gathering, literature review, system analysis, system design, implementation, testing, and documentation. Each phase was assigned a specific timeline and milestones to ensure steady progress throughout the project lifecycle. The technologies used for development were familiar to the development team, which reduced the learning curve and minimized potential delays. Regular monitoring of project activities and effective coordination among team members helped ensure that tasks were completed according to schedule. Although minor challenges were encountered during development and testing, they were resolved within the planned timeframe. Therefore, considering the available resources, team capabilities, and project scope, the Stock Smart system was successfully completed within the designated schedule, demonstrating its schedule feasibility.

3.1.3 Data Modeling

Data modeling is the process of defining the structure of data and relationships among system entities. The Stock Smart system utilizes an Entity Relationship (ER) model to represent data organization.

The primary entities within the system include:

- User
Stores information about system users including user ID, username, email, password, and role.
- Product
Contains product details such as product ID, product name, SKU, category, quantity, purchase price, selling price, and stock status.
- Category
Stores category information used to classify products into logical groups.
- Supplier
Contains supplier-related information including supplier name, contact details, and address.
- Sales
Stores sales transaction information including transaction ID, date, customer information, and total amount.
- Sales Details
Records individual products associated with each sales transaction.
- Inventory Movement
Tracks stock additions, reductions, transfers, and adjustments.
Relationships among entities ensure data consistency and support efficient inventory management operations

3.1.4 Process Modeling

Process modeling describes how data flows through the system and how users interact with various functionalities.

- Login Process
Users enter authentication credentials. The system validates user information and grants access according to assigned roles.
- Product Management Process
Authorized users add, update, view, or delete product records. Changes are immediately reflected in the inventory database.
- Inventory Update Process

Whenever inventory quantities change due to purchases, sales, or adjustments, the system updates stock records and logs inventory movements.

- **Sales Process**

The cashier selects products and completes a sales transaction. Inventory quantities are automatically deducted, and sales records are stored in the database.

- **Report Generation Process**

Users select report parameters. The system retrieves relevant data and generates inventory or sales reports for analysis and decision-making.

3.2 System Design

In the third step, the overall structure of StockSmart is designed before any coding begins. This includes the design of the database schema, which defines how data such as product information, transaction records, and user details are stored and related to one another. User flow diagrams are created to map out how different types of users, namely the Admin, Inventory Manager, and Cashier, navigate and interact with the system. User interface wireframes are also developed to plan the layout and visual structure of each screen. Additionally, the core modules of the system are defined at this stage, including the Inventory module, Alert module, Dashboard, and Reports module. This design phase ensures that the system is well-organized and that all components are planned coherently before implementation begins.

3.2.1 Architectural Design

Stock Smart follows a three-tier architecture consisting of Presentation Layer, Application Layer, and Database Layer.

- **Presentation Layer**

The presentation layer provides the user interface through which users interact with the system. It includes dashboards, forms, reports, and inventory management screens.

- **Application Layer**

The application layer contains business logic responsible for processing requests, validating data, enforcing access control, and managing inventory operations.

- **Database Layer**

The database layer stores system data including user accounts, products, inventory records, suppliers, and transaction histories.

This architecture improves maintainability, scalability, security, and separation of concerns.

Figure 2: Cashier

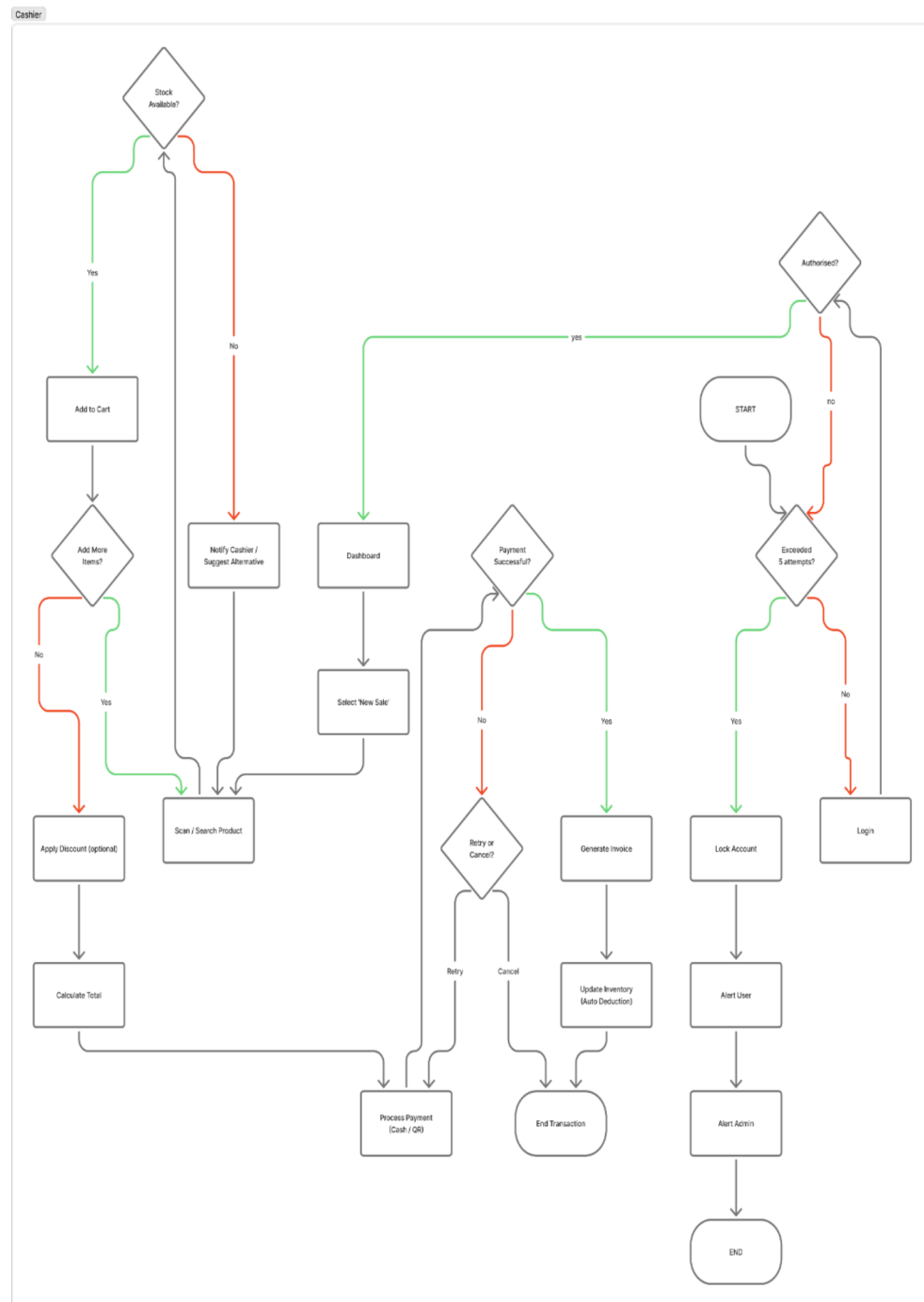
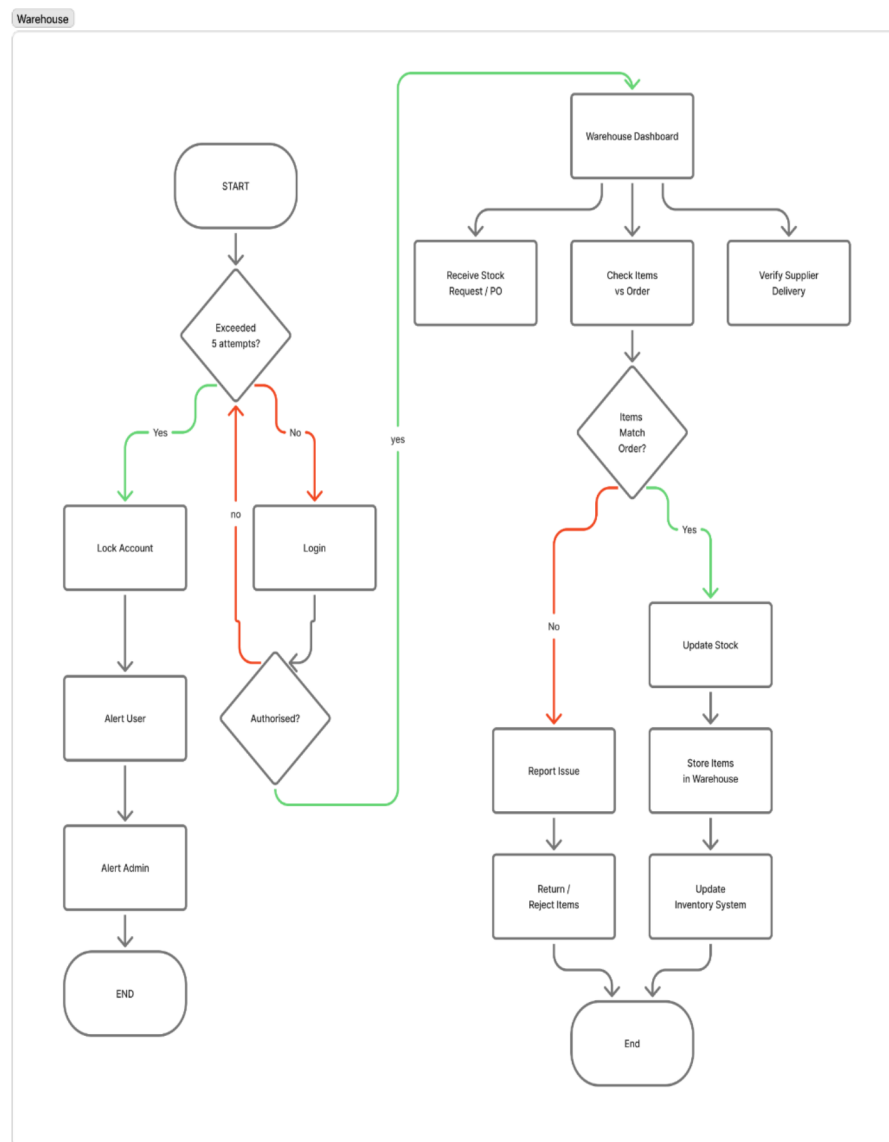


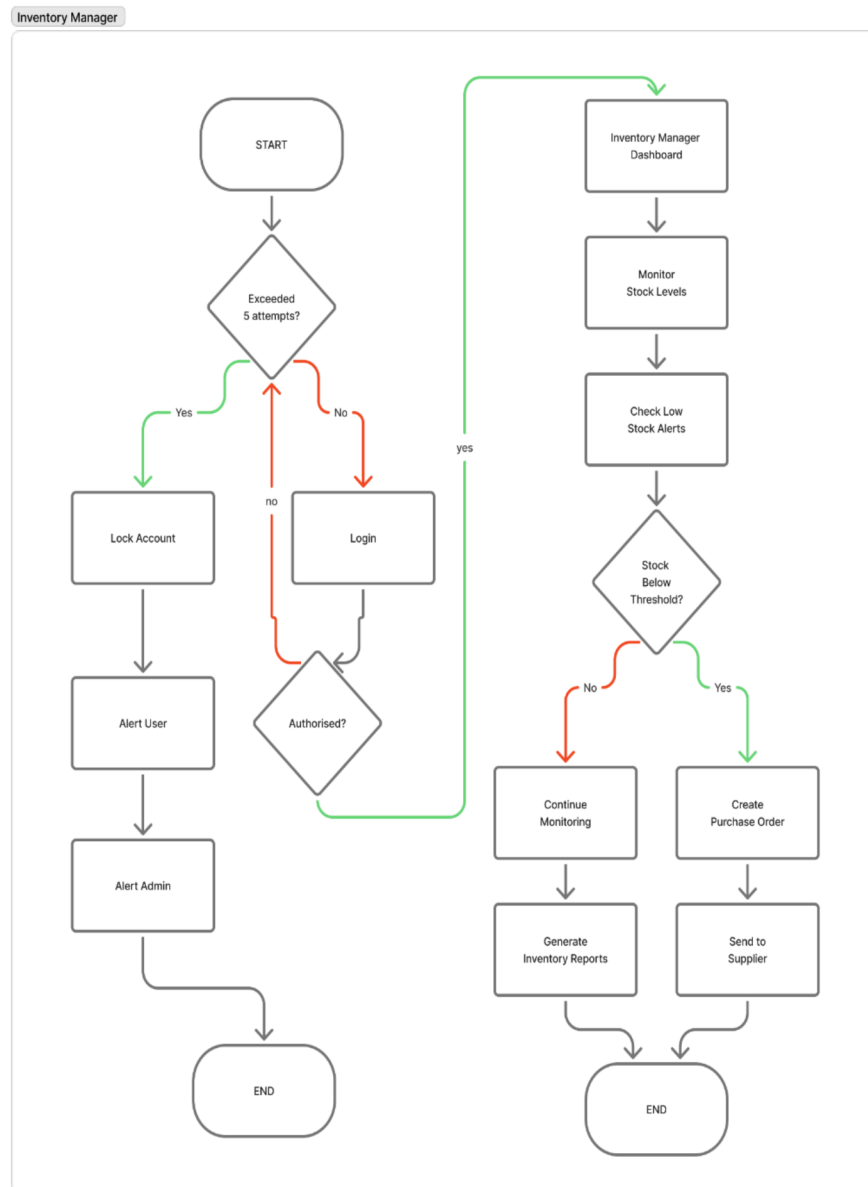
Figure 2 illustrates the cashier workflow in the StockSmart system. The process begins with user authentication, where the cashier logs in and gains access to the dashboard after successful authorization. The cashier then searches for products, verifies stock availability, adds items to the cart, applies discounts if necessary, and calculates the total amount before processing the payment. Upon successful payment, the system generates an invoice, automatically updates inventory records, and completes the transaction, while unsuccessful payments allow the cashier to retry or cancel the transaction.

Figure 3: Warehouse



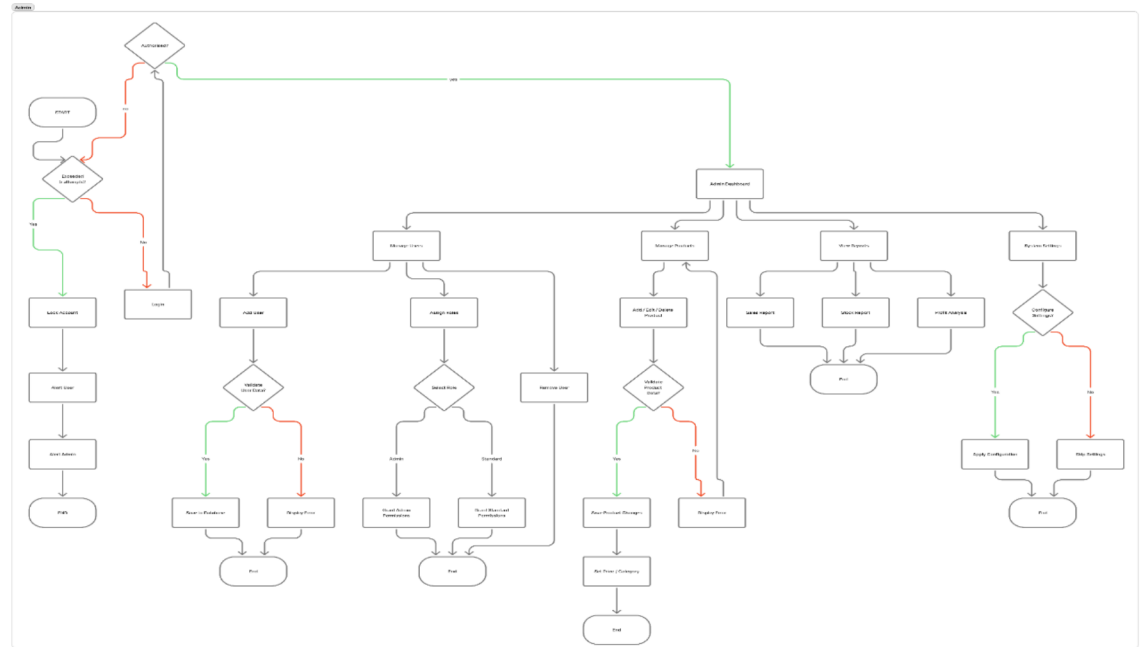
The warehouse flowchart illustrates the process of user authentication and inventory management within the system. Users must log in and are granted access only after successful authorization, while accounts are locked and notifications are sent after exceeding five failed login attempts. Once authenticated, warehouse staff can receive purchase orders, verify supplier deliveries, and compare received items with the order details. If the items match, the stock and inventory system are updated; otherwise, issues are reported and the incorrect items are returned or rejected.

Figure 4: Inventory Manager



The inventory manager flowchart illustrates the process of user authentication and inventory monitoring within the system. Users are granted access only after successful authorization, while accounts are locked and notifications are sent to both the user and administrator after five failed login attempts. Once logged in, the inventory manager monitors stock levels and checks for low stock alerts. If stock falls below the threshold, a purchase order is created and sent to the supplier; otherwise, the system continues monitoring and generates inventory reports.

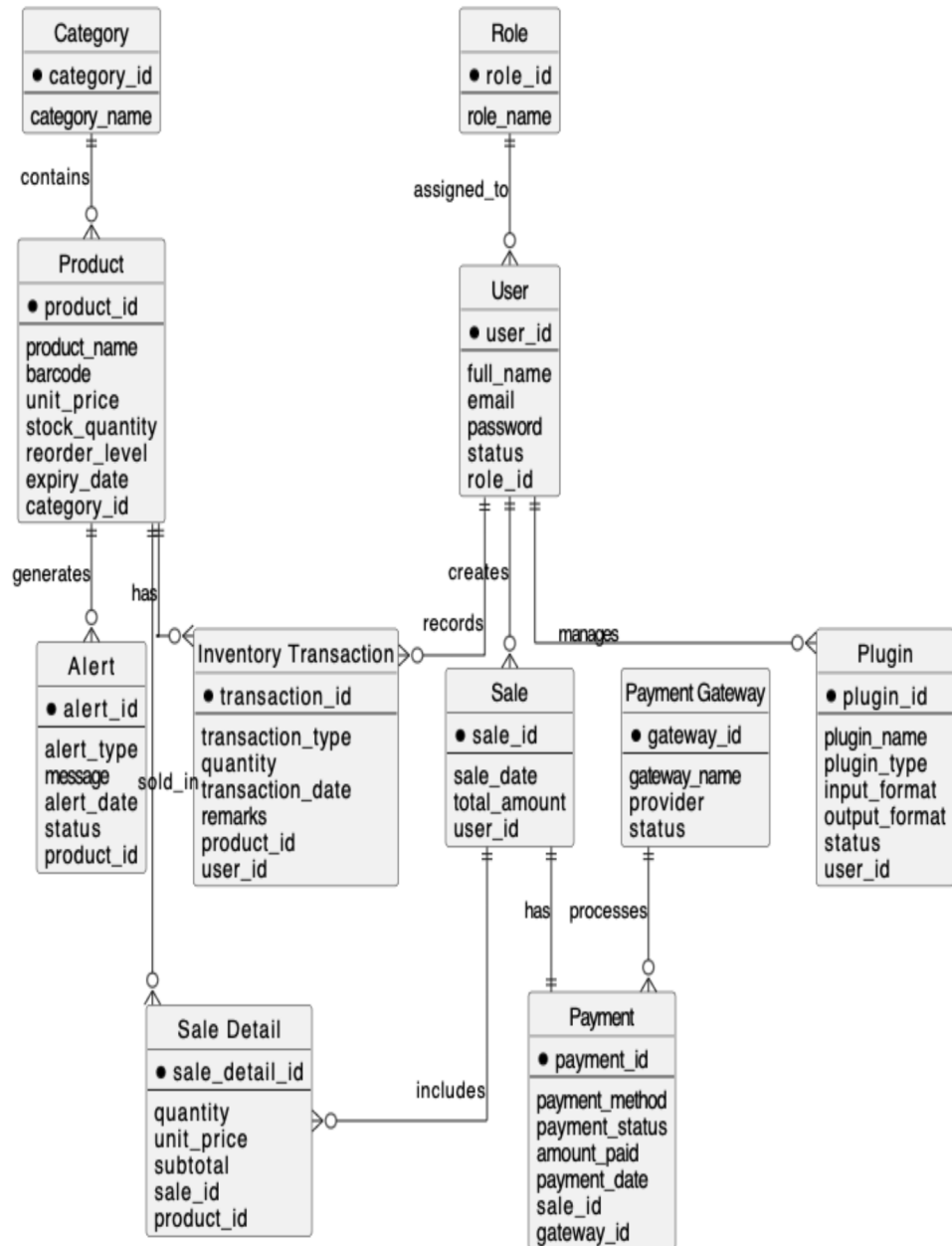
Figure 5: Admin



The admin flowchart illustrates the process of user authentication and the management of key system functions. After successful authorization, the administrator can manage users, inventory products, reports, and system settings, while accounts are locked and notifications are issued after five failed login attempts. The administrator can approve or reject user accounts and product changes, generate and view reports, and configure system preferences. These functions ensure secure access, effective system administration, and smooth operation of the inventory management system.

Figure 6: ER Model

StockSmart ER Model



The StockSmart ER model represents the relationships between the entities involved in the inventory management system. Users are assigned roles and are responsible for creating sales, recording inventory transactions, and managing plugins. Products are organized into categories and are associated with inventory transactions, sales details, and alerts that are generated when stock conditions require attention. Sales consist of multiple sale details and are linked to payments,

which are processed through payment gateways, enabling efficient management of inventory, sales, and payment operations within the system.

3.2.2 Database Schema Design

The database schema consists of several related tables:

- Users
- Roles
- Products
- Categories
- Suppliers
- Inventory
- Sales
- Sales_Details
- Inventory_Movements

Primary keys uniquely identify records, while foreign keys establish relationships between tables. Database normalization techniques are applied to minimize redundancy and improve data integrity.

3.2.3 Interface Design

The user interface was designed to provide simplicity, accessibility, and efficiency.

Major interfaces include:

- Login Interface
Allows users to authenticate and access the system securely.
- Dashboard Interface
Displays inventory statistics, stock alerts, recent transactions, and summary information.
- Product Management Interface
Enables users to manage product records and inventory information.
- Inventory Interface
Provides inventory monitoring, stock updates, and inventory movement tracking.
- Sales Interface

Facilitates sales transaction processing and inventory deduction.

- Reports Interface

Generates sales, inventory, and stock movement reports.

3.2.4 Physical Model

The physical model describes how the system is deployed within the computing environment.

The application is hosted on a web server and accessed through client devices using a web browser. User requests are transmitted through the network to the application server, where business logic is executed. The application server communicates with the database server to retrieve and store information.

Hardware requirements include:

- Processor: Intel Core i3 or equivalent
- RAM: Minimum 4 GB
- Storage: 20 GB available space
- Stable internet connection

Software requirements include:

- Operating System: Windows, Linux, or macOS
- Web Browser: Chrome, Firefox, Edge, or Safari
- Database Management System: MySQL
- Web Server: Apache or equivalent server environment

This deployment model provides centralized data management, easy maintenance, and accessibility from multiple locations.

Chapter 4: Implementation and Testing

The final step involves the actual construction and evaluation of the StockSmart system. Development is carried out module by module, following the design specifications established in Step 3. Once each module is built, it undergoes individual testing to verify that it functions correctly and meets the requirements identified in Step 1. After all modules are integrated, the complete system is tested for overall performance, accuracy, and reliability. Validation is carried out using sample data derived from realistic SME scenarios to ensure that the system behaves as expected under conditions that reflect its intended use environment. Any issues identified during testing are addressed before the system is considered complete.

4.1 Implementation

4.1.1 Tools Used

Various software tools were utilized during the development of the Stock Smart inventory management system. These tools assisted in system design, coding, testing, database management, and documentation.

- Visual Studio Code

Visual Studio Code (VS Code) was used as the primary source code editor for developing the application. It provided features such as syntax highlighting, debugging support, code completion, and extension integration, which improved development efficiency.

- Git and GitHub

Git was used as the version control system to track changes in the source code and manage project development. GitHub served as the repository hosting platform, enabling collaboration among team members and maintaining backup copies of the project.

- MySQL Workbench

MySQL Workbench was used for database design, schema creation, query execution, and database administration. It facilitated the visualization and management of database structures.

- Draw.io

Draw.io was used to create system diagrams including Entity Relationship Diagrams (ERD), flowcharts, architectural diagrams, and process models.

- **Microsoft Word**

Microsoft Word was used for preparing project documentation, report writing, formatting, and final report compilation.

- **Web Browsers**

Modern web browsers such as Google Chrome and Mozilla Firefox were used for application testing, debugging, and user interface verification throughout the development process.

4.1.2. Test Cases for system Testing

System testing was conducted to ensure that all major functionalities of the Stock Smart system operated correctly and met the specified requirements. Various test cases were performed, including user authentication, product management, inventory updates, sales transaction processing, report generation, and role-based access control. The testing process verified that users could successfully log into the system, manage product information, update inventory records, and generate reports without errors. Invalid login attempts were also tested to ensure that unauthorized users could not access the system. Additionally, stock quantities were checked after sales transactions to confirm that inventory levels were updated accurately. The results of the testing indicated that the system performed as expected, with all critical functionalities operating correctly and reliably.

Chapter 5: Conclusion and Future Recommendations

5.1 Lessons Learned

The development of Stock Smart provided valuable practical experience in software engineering, system analysis, database design, and web application development.

Through this project, the team gained a deeper understanding of inventory management processes and the challenges faced by small and medium-sized enterprises in maintaining accurate stock records. The project enhanced technical skills related to programming, database management, user interface design, testing, and debugging. Team members also learned the importance of proper requirement analysis, project planning, and effective communication during software development. Furthermore, the implementation and testing phases highlighted the significance of data accuracy, system security, and user-friendly design in developing reliable business applications. Overall, the project served as a valuable learning experience that strengthened both technical knowledge and teamwork skills while providing practical exposure to real-world software development practices.

5.2. Conclusion

StockSmart is expected to become a simple, efficient, and affordable inventory management system for SMEs. The project focuses on solving common inventory problems such as stock shortages, overstocking, expired products, and manual record errors. By providing real-time inventory tracking, automated alerts, and organized digital records, StockSmart can help businesses improve efficiency, reduce operational costs, and make better inventory decisions. The expected outcomes, benefits, and impacts of the project demonstrate that StockSmart has strong practical value and real-world business applications. With its affordable implementation and user-friendly design, the project can become an effective solution for modern inventory management.

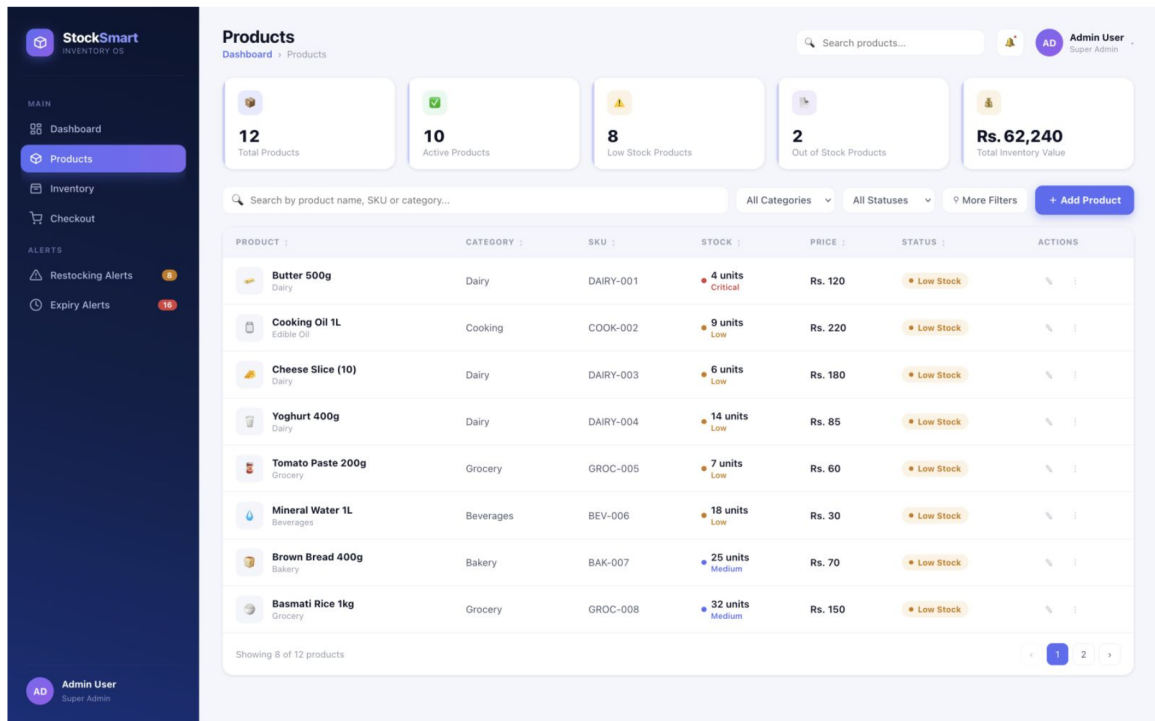
5.3. Recommendations

Although StockSmart successfully fulfills its intended objectives as an inventory management system for SMEs, several improvements can be considered for future development. The system can be enhanced through the integration of barcode and QR code scanning technologies to improve inventory accuracy and reduce manual data entry. A mobile application could also be developed to provide users with greater accessibility and remote inventory management capabilities. Future versions may incorporate advanced reporting and analytics features, including data visualization

and demand forecasting, to support more informed business decisions. The addition of supplier and purchase management modules would further streamline procurement processes, while support for multi-branch inventory management would enable larger organizations to manage stock across multiple locations. Integration with accounting systems and cloud-based deployment options could improve operational efficiency and accessibility. Furthermore, the implementation of artificial intelligence techniques for demand prediction, along with enhanced security features such as two-factor authentication and audit logging, would increase the system's effectiveness and reliability. Finally, broader user acceptance testing and long-term deployment in real business environments are recommended to evaluate system performance and identify opportunities for continuous improvement.

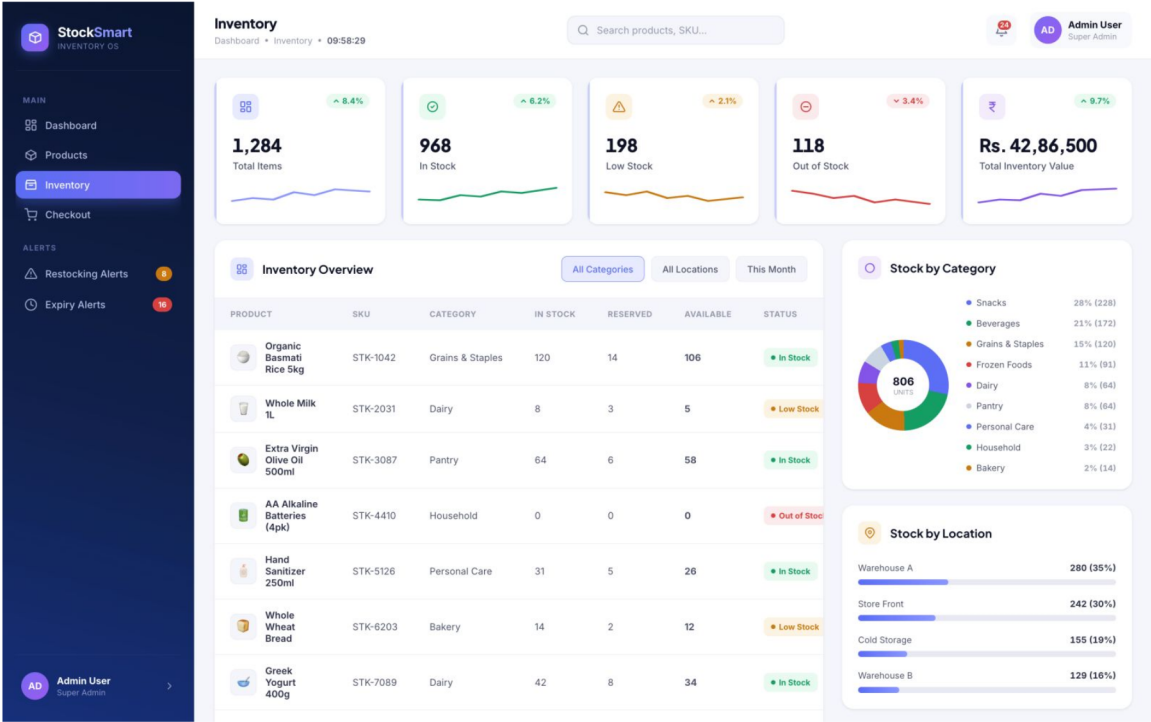
Appendices

Figure 7: Products page



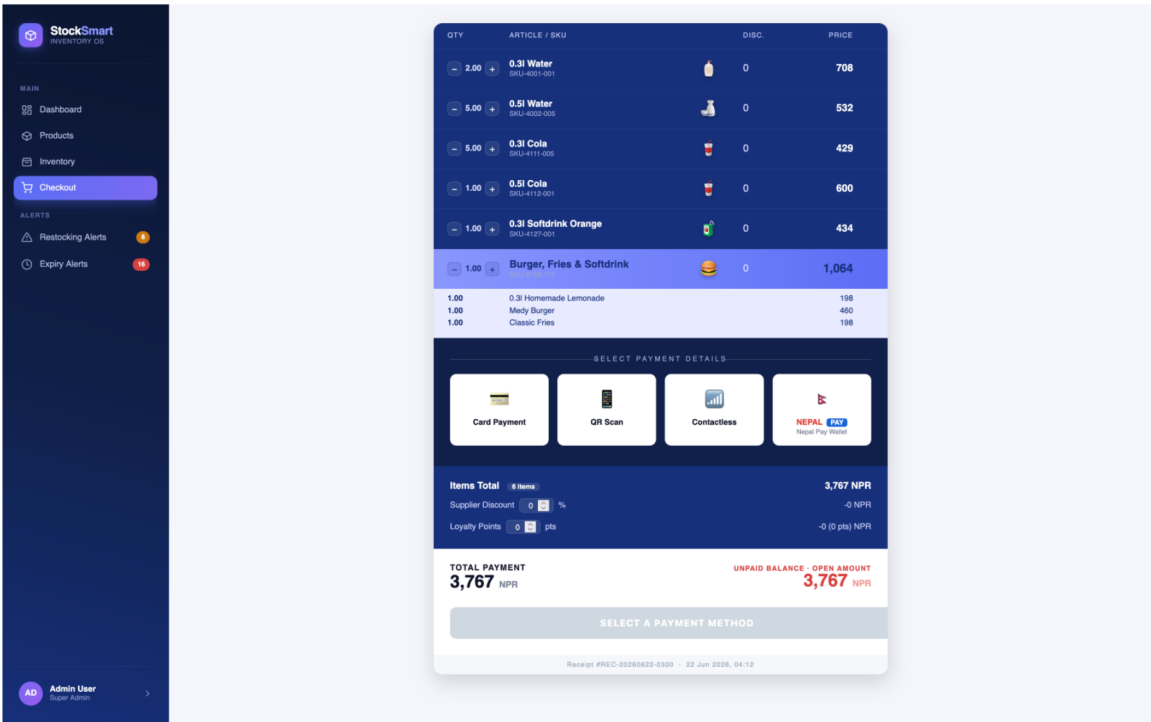
This product display page was designed in Figma and serves as the main interface for managing inventory products. It provides a clear overview of key inventory statistics, including total products, active products, low-stock items, and total inventory value. Users can easily search, filter, and manage products through an organized and user-friendly layout. The product table displays important details such as product name, category, SKU, stock level, price, and status. Overall, the page enhances inventory management by providing quick access to product information and administrative actions.

Figure 8: Inventory page



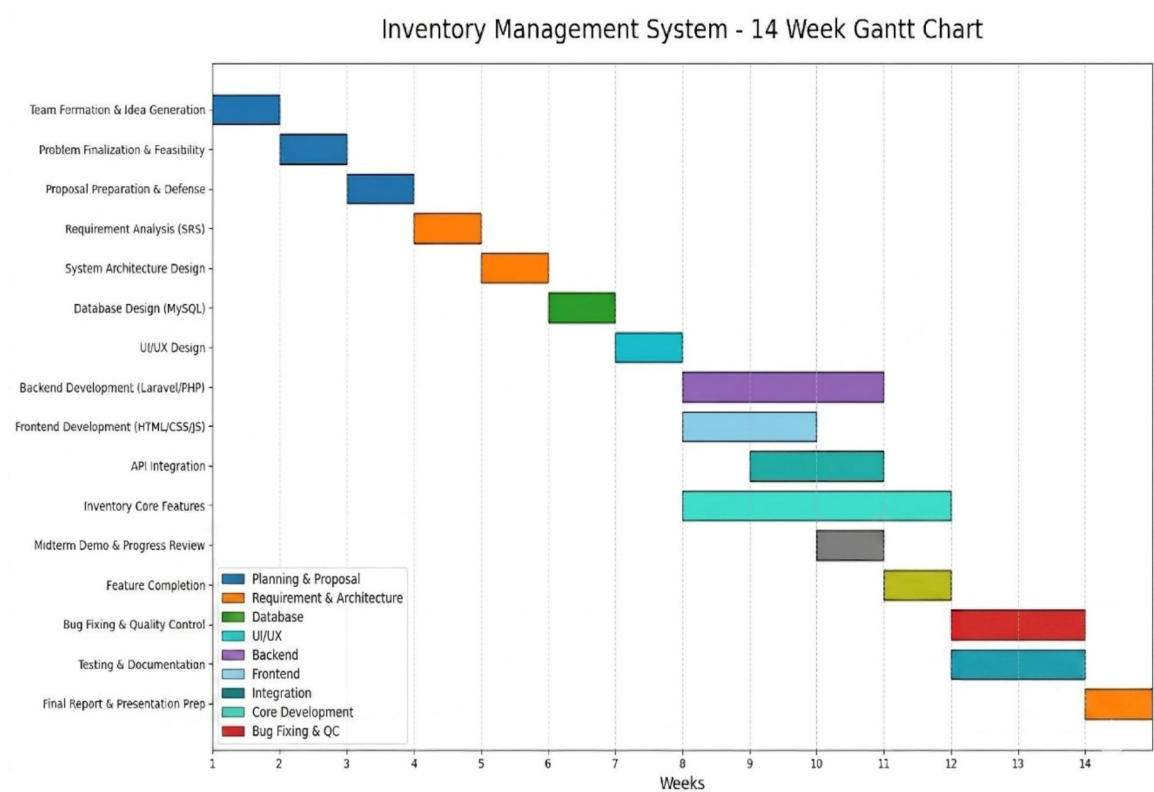
This inventory page was designed in Figma and serves as the central hub for monitoring and managing inventory across different locations. It provides a quick overview of key inventory metrics, including total items, items in stock, low-stock products, out-of-stock products, and total inventory value. The page features an inventory overview table displaying product details, stock availability, reserved quantities, and storage locations. Visual charts and graphs help users analyze stock distribution by category and location for better decision-making. Overall, the interface offers an organized and efficient way to track inventory levels and maintain stock control.

Figure 9: Checkout page



This checkout page was designed in Figma to provide a smooth and efficient point-of-sale experience. It displays a detailed summary of selected products, including quantities, item descriptions, discounts, and total prices. Multiple payment options, such as card payment, QR scan, contactless payment, and digital wallets, are available to offer flexibility to customers. The page also presents billing information, payment status, and the final amount due in a clear and organized format. Overall, the interface ensures a fast, user-friendly, and secure checkout process for both customers and staff.

Figure 10: Gantt Chart



This 14-week Gantt chart illustrates the timeline and sequence of activities for the development of the Inventory Management System. The project begins with the planning and proposal phase, including team formation, problem finalization, and proposal defense during the first three weeks. This is followed by requirement analysis, system architecture design, database design, and UI/UX design to establish the system structure and user interface.

From Week 8 onward, the main development phase starts, where backend and frontend development are carried out simultaneously along with API integration and the implementation of core inventory features. A midterm progress review and feature completion are conducted before entering the final stages of bug fixing, quality control, testing, and documentation. The project concludes in Week 14 with the preparation of the final report and presentation, ensuring that the system is fully tested and ready for deployment.

References

- Abdul Majid, K., & Kavyashree, G. J. (2025). *Smart inventory and sales analytics for supermarkets*.
- Agboola, F. F., Malgwi, Y. M., Mahmud, M. A., & Oguntoye, J. P. (2022). *Development of a web-based platform for automating inventory management of SMEs*.
- Haider, G., & Lucas, E. (2024). *Smart inventory management for SMEs using IoT and automation*.
- Laudon, K. C., & Laudon, J. P. (2021). *Management information systems: Managing the digital firm*. Pearson.
- Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach (9th ed.)*. McGraw-Hill.
- Qureshi, M. R. N. M. (2022). *Evaluating enterprise resource planning (ERP) implementation for sustainable supply chain management*.
- Sommerville, I. (2016). *Software engineering (10th ed.)*. Pearson.